

Concept and variable-template template-parameters

Document #:	P2841R7
Date:	2025-02-14
Programming Language C++	
Audience:	CWG
Reply-to:	Corentin Jabot < corentin.jabot@gmail.com > Gašper Ažman < gasper.azman@gmail.com > James Touton < bekenn@gmail.com > Hubert Tong < hubert.reinterpretpcast@gmail.com >

Abstract

C++ allows passing templates as template parameters. However, they are forced to be type-names (either type alias templates or class templates). Variable templates or concepts are not supported. This is a hole in the template facilities and is the topic of this paper.

We introduce a way to pass concepts and variable templates as template parameters.

Example:

```
template<
    template <typename T> concept C,
    template <typename T> auto V
>
struct S{};

template <typename T>
concept Concept = true;

template <typename T>
constexpr auto Var = 42;

S<Concept, Var> s;
```

Note: this paper is a subset of the larger [P1985R3](#) [1] (Universal Template Parameters); the authors felt this topic is subtle enough to warrant its own paper.

Revisions

R7

- The consistent usage of *template-parameter* and template parameter has been [addressed editorially](#), and the wording rebased.

- The renaming from non-type template parameter to constant template parameter has been [addressed editorially](#), and the wording rebased.
- Core review in Hagenberg.

Made fold expressions containing both concepts template parameters and non-concept template parameters ill-formed:

```
template <typename T, template <typename> concept>
struct wrapper{};

template <typename... T, template <typename> concept... CTs>
int f(wrapper<T, CTs>...) requires (CTs<T> && ...); // \#3

int err = f(wrapper<int, A>{}, wrapper<int, B>{}); // error: the fold expression in \#3
expands mixed kind template parameters.
```

R6

- Specify that a concept template parameters cannot be constrained
- Specify that a a concept-id is value dependent if its concept-name is dependent
- Specify that a template template parameter cannot be constrained by a concept-dependent constraint to address the following scenario

```
template <
    template <typename> concept C,
    template <C> class TT
>
struct A {
    static int x;
};

template <
    template <typename> concept C,
    template <C> class TT
>
int A<C, TT>::x = 0; // A cannot be named
```

R5

- Following CWG review and concertation on the EWG reflector, concept template parameters appearing in an enclosing template are not eligible for subsumption.
- Wording fixes.

R4

- Templated functions referring to template concept parameters are no longer eligible for subsumption.

- Improve the constraint normalization section.
- Improve and add examples
- Rename the new grammar production (*variable-tt-parameter*, *type-tt-parameter*, *concept-tt-parameter*).
- Use the prose terminology and typography in more places
- Improve the sections introducing template parameters and pack thereof
- Change the grammar of template arguments not to use id-expression
- Introduce "concept dependent constraints"
- Address CWG feedback.

R3

- Wording improvements

R2

- Alter the design so that partial ordering remains independent of template arguments, following guidance given in Kona.
- Add a section on the deduction of template parameters from the arguments of a variable template/concept specialization

R1

- Add examples, motivation
- Wording improvement

R0

- Initial revision

Motivation

Template template-parameters allow for higher-order templates and greater composability. They can be used, for example, to parametrize a function that operates on any container of any type or to write CRTP-based interfaces.

C++23 limits template template-parameters to be class templates or an alias templates. A variable template (C++14) or a concept (concepts are themselves templates) cannot be passed as a template argument in C++23.

The motivation for passing a concept as a template argument is very much the same as our reason for supporting class templates as template arguments: to allow higher-level constructs.

While there are workarounds - for example by wrapping a variable in a struct with a `value` member that can then be passed as a type template parameter, these workarounds all suffer the same limitations:

- They have terrible ergonomics
- They have a noticeable impact on performance - instantiating types is expensive
- They do not allow to take advantage of nice concept properties such as terse syntax and subsumption.

All of these limitations of available patterns are additional motivations for this proposal.

Being able to define a concept adaptor, for instance, would be very nice:

```
template <typename T, template <typename> concept C>
concept decays_to = C<decay_t<T>>;
```

Being able to use it with any concept constraint would also be helpful:

```
template <decays_to<copyable> T>
auto f(T&& x);
```

Other such constructs might, for example, include the following.

- `range_of<Concept>`
Many algorithms can operate on a sequence of integer or string-like types and while it is possible to express `range<T> && SomeConcept<ranges::range_reference_t<R>>`, some codebases do that enough that they might want to have a shorter way to express that idea, one that would let them use the abbreviated syntax in more cases.
- `tuple_of<Concept>`
This follows the same idea, but expressing this idea in the `require` clause of each function or class that might need it would be an exercise in frustration and a maintenance nightmare. We explore a `tuple_of` concept later in this paper. Representing vectors as tuples-like things of numbers is common in the scientific community, and these scientific libraries have no ideal way to express these constraints.
- Avoiding duplication.
In his [blog post](#) on this very topic, Barry Revzin observed that `std::ranges` defines a handful of concepts that are very similar to one another except they use different concepts internally. Concept template parameters can reduce a lot of duplication. Compare the [definitions in the Standard](#) and [the implementation with our proposal](#).

To quote Barry's aforementioned blog post

I'd rather write a one-line definition per metaconcept, not a one-line definition per metaconcept instantiation.

So part of the motivation for concept template-parameters is the same as for having functions, templates, and classes: We want to be able to reuse code and to make it less repetitive and error-prone.

We also demonstrated how this feature can be leveraged to provide better diagnostics when a concept is not satisfied [\[Compiler Explorer\]](#).

There is community interest in these features.

- [Is it possible to pass a concept as a template parameter?](#)
- [Concept to assert an argument is another concept, with whatever parameters](#)
- [Passing a concept to a function](#)
- [How to pass a variable template as template argument](#)
- [Can a variable template be passed as a template template argument?](#)

Unfortunately, this is one of those features that truly shows its power on large examples that don't tend to fit into papers.

Variable-template template-parameters

Variable-template template-parameters (previously proposed in [P2008R0 \[6\]](#)) are useful by themselves. They can be emulated with a template class with a static public `value` data member. Most standard type traits are defined as a type and have an equivalent `_v` variable:

```
template <typename T, typename U>  
constexpr bool is_same_v = is_same<T, U>::value;
```

But this is not compile-time efficient: a class has to be instantiated in addition to generating the value for the constant, which is strictly more work than just producing the constant. For a 'bool' constant, for instance, the difference is substantial; on Apple clang15, it's about 60% (so, less than half the time). The memory footprint is more difficult to gauge, but it seems around a 40% difference.

This performance issue is also explored in more detail in [P1715R1 \[2\]](#).

In other microbenchmarks, Gašper has observed a minimum of 30% speedup by not instantiating class bodies, and a 50% memory usage reduction for programs with heavy traits usage, specifically when implementing P2300-like classes.

Also, if one has multiple metaprogramming libraries, relying on idioms like `::value` is fundamentally less composable than a value just being a value. Similarly, if you have a concept in your codebase, you shouldn't have to wrap it into a static `constexpr ::value` member of a type to pass it to a metafunction.

Wrapping variables in class templates also adds complexity for users: The main reason we expose both a variable template and a class template for every boolean trait is that the language does not support variable-template template-parameters. (Note that we are aware of some codebases using traits as tags for dispatch but this is far from the common case.)

For instance, counting elements `Ts` that satisfy a specific predicate `p` could be done as

```
template <template <typename> auto p, typename... Ts>  
constexpr std::size_t count_if_v = (... + p<Ts>);
```

We could do the same thing with a type, but it incurs a class template instantiation for each element:

```
template <template <typename> typename p, typename... Ts>  
constexpr std::size_t count_if_v = (... + p<Ts>::value);
```

It will always be more work for the compiler to instantiate a whole class together with its body (not just its declaration) to allow access to the inner value member than just instantiating a variable template, no matter how much we try to optimize this pattern. [p1715r1](#) [?] makes the same case.

Additional examples

The authors have use cases that don't fit in the paper (typical for the most *interesting* use cases) where type-based vs variable-based metaprogramming means the difference of 300s compile-times per unit vs. more than an hour (currently by textually duplicating definitions that could have been genericized if variable template template-parameters were available).

Terse syntax, overloading, and reusing existing concepts

The following example, simplified from production code shows multiple interesting properties of concept template parameters.

Here we demonstrate the algorithm `with_values_t` with `optional` and `expected`. Its call operator applies `f` to all engaged arguments; however, all the arguments must be of the same shape (all `optionals`, all `expected`), etc.

To do that, we here use the abbreviated function template syntax with type-constraints, which is only possible with concept template parameters.

```
template <typename T> struct optional;
template <typename V, typename E> struct expected;

template <typename> constexpr bool _some_maybe_v = false;
template <typename T> constexpr bool _some_maybe_v<optional<T>&> = true;
template <typename T> constexpr bool _some_maybe_v<optional<T> const&> = true;
template <typename> constexpr bool _some_expected_v = false;
template <typename T, typename E> constexpr bool _some_expected_v<expected<T, E>&> =
    true;
template <typename T, typename E> constexpr bool _some_expected_v<expected<T, E> const&> =
    true;

template <typename T>
concept some_maybe = _some_maybe_v<T&>;
template <typename T>
concept some_expected = _some_expected_v<T&>;

template <template <typename> concept C>
struct _with_values_t {
    static constexpr auto operator()(auto&& f, C auto&& v, C auto&& ... vs) -> decltype(auto) {
        if constexpr (sizeof...(vs) == 0) {
            if (v) {
                return f(*FWD(v));
            } else {
                return f();
            }
        } else {
            if (v) {
                return _with_values_t{}(bind_front(f, *FWD(v)), FWD(vs)...);
            } else {
                return _with_values_t{}(f, FWD(vs)...);
            }
        }
    }
};

// Enforce it's the same monad
inline constexpr struct with_values_t : _with_values_t<some_maybe>, _with_values_t<some_either>
{
    using _with_values_t<some_maybe>::operator();
    using _with_values_t<some_either>::operator();
};
```

```
} with_values {};
```

It would be technically possible to use a type instead here

```
template <typename T>
struct some_expected_t {
    static constexpr bool value = some_expected<T>;
};

struct _with_values_t {
    template <typename First, typename... Tail>
        requires (some_expected_t<First>::value && (some_expected_t<Tail>::value && ...))
    static constexpr auto operator()(auto&& f, C auto&& e, C auto&& ... es) -> decltype(auto);
};
```

But again:

- This is much less ergonomic as it forces users to wrap their concepts in types which is not intuitive (ie we have found that difficult to teach).
- The necessity of introducing new names for the same predicate - just exposed as a type, concept, or variable - adds unnecessary complexity to APIs
- Composability only works by convention.
- Creating types has a significant performance impact on compile times
- Diagnostic messages are slightly worse than they could be because of the added layers of wrapping and because compilers will decompose concepts in diagnostic messages.

A variant of this function is used in an implementation of the `upon_all_done(handle_successes, handle_failures, senders...)` sender transformer in one internal codebase, where `handle_successes` is invoked with all the successful completions, and `handle_failures` with all errors.

When life gives you Lambdas

To work around the lack of concept parameters, users have started to use generic lambdas

```
template <typename T, auto ConceptWrapperLambda>
concept decays_to = requires {
    ConceptWrapperLambda.template operator()<std::decay_t<T>>>();
};

template <class T>
requires decays_to<T, ([<std::copyable>(){}]>>
auto f(T&& x) {}
```

Here the concepts we want to parametrize on are passed as a constrained generic lambda - which we then try to call when checking our higher-level concepts. This allows not to have to create a new type for each concept, so it might be slightly easier to use, although the reader will agree that it particularly arcane. In addition to the usability concerns, lambdas are never a solution to compile times performance.

All the existing work arounds suffer similar performance and usability concerns, and of course none support subsumption. Yet, many such workarounds have been developed and a number of them have been deployed in production. Daisy Hollman provided an [entire collection of such workarounds](#).

Previous work

Variable-template template-parameters were proposed in [P2008R0](#) [6] and were part of the original design for variable templates [N3615](#) [7]. Concept template-parameters have been described by Barry Revzin (back when Concept names were uppercase) in his blog [here](#) and [here](#). We mentioned them in [P2632R0](#) [5] and [P1985R3](#) [1].

Universal template-parameters

The fact that variable-template template-parameters and concept template-parameters appear in the same papers is not accidental. For a universal template-parameter to be universal, we need to make sure it covers the set of entities we could want to use as template-parameters. There is, therefore, an important order of operations. If we were to add universal template-parameters before concept template-parameters and variable-template template-parameters, we would be in a situation where either

- we can't ever add concept template-parameters and variable-template template-parameters
- "universal template-parameters would not be truly universal"
- we would feel forced to come up with some kind of "more universal template-parameter" syntax

None of these outcomes seems desirable; therefore, the best course of action is to ensure that we support as best we can the full set of entities we might ever want to support as template-parameters, before adding support for universal template-parameters.

Design

Syntax

We propose the following syntax for the declaration of a template head accepting a concept as a parameter:

```
template<
    template <template-parameter-list> concept C
>
```

We propose the following syntax for the declaration of a template head accepting a variable template as parameter:

```
template<
    template <template-parameter-list> auto C
>
```

Note that because variable templates and their type can be arbitrarily specialized, `auto` here acts only as a syntactic marker and cannot be replaced by a *type-id*.

This forms a natural, somewhat intuitive extension of the existing syntax for template extension:

```

template<
    typename T,
    auto V,
    template <template-parameter-list> typename TT,
    template <template-parameter-list> auto VT,
    template <template-parameter-list> concept C,
>

```

Default Arguments

Like type template parameters, concepts, and variable template parameters can have a default argument that is a concept name or the name of a variable template respectively. Packs can't be defaulted. (That's a separate paper!)

Usage

Within the definition of a templated entity, a concept template-parameter can be used anywhere a concept name can be used, including as a type constraint, in the requires clause, and so forth.

For example, the following should be valid:

```

template <template <typename T> concept C>
struct S {
    void f(C auto);
};

```

Concept template-parameters and subsumption

Consider:

```

template <typename T>
requires view<T> && input_range<T>
void f(); // #1

template <typename T>
requires view<T> && contiguous_range<T>
void f(); // #2

```

We expect #2 to be more specialized than #1 because `contiguous_range` subsumes `input_range`.

Now, consider:

```

template <typename T>
requires all_of<T, view, input_range>
void f(); // #1

template <typename T>

```

```
requires all_of<T, view, contiguous_range>
void f(); // #2
```

[\[Run this example on Compiler Explorer\]](#)

This example ought to be isomorphic to the previous one, and #2 should still be more specialized than #1. To do that, we need to be able to substitute concept template arguments in constraint expressions when normalizing constraints.

When establishing subsumption, we have historically not substituted template arguments, instead establishing a mapping of template parameters to arguments for each constraint and comparing those mappings.

But to establish subsumption rules for concept template-parameters, we need to depart from that somewhat.

Concepts have the particularity of never being explicitly specialized, deduced, dependent, or even instantiated. Substituting a concept template argument is only a matter of replacing the corresponding template parameter with the list of constraints of the substituted concept, recursively.

As such, subsumption for concept template-parameters does not violate the guiding principle of subsumption.

For example, a `range_of_integrals` defined as follow:

```
template<typename T>
concept range_of_integrals = std::ranges::range<T> && std::integral<std::remove_cvref_t<std::ranges::range_reference_t<T>>>>;
```

Can be mechanically lifted:

```
template<typename T, template <typename...> concept C>
concept range_of = std::ranges::range<T> && C<std::remove_cvref_t<std::ranges::range_reference_t<T>>>>;
template<typename T>
concept range_of_integrals = range_of<T, std::integral>;
```

Note that this transformation does not change any other behavior of normalization, i.e., concept template-parameters that appear within other atomic constraints are not substituted, and arguments that are not concept names are not substituted either.

Fold expressions involving concept template-parameters

Our proposed design allow for subsumption in the the presence of fold expressions whose pattern is a concept. (For the non-concept case, see [P2963R0 \[4\]](#))

```
template <
    typename T,
    template <typename...> concept... C>
concept all_of = (C<T> && ...);
```

Once substituted, the sequence of binary && or || is normalized, all_of, any_of, and so on can then be implemented in a way that supports subsumption.

One very important case where this facility is absolutely essential is constraining tuples (and other algebraic data-types) by dimension:

```
template <typename X, template <typename> concept... C>
concept product_type_of = (... && C<std::tuple_element_t<C...[?], X>>);
//   index-of-current-element, not proposed, but needed ~~~~~~
```

[P2632R0](#) [5] discusses alternatives to the awful index-of-current-element syntax above.

ADL

Similar to variables, variable templates and concepts are not associated entities when performing argument-dependent lookup. This is consistent with previous work (for example [N3595](#) [3] and [P0934R0](#) [8]) and the general consensus toward ADL.

Deduction of concept and template parameters

Variable and concept template-parameters should be deducible from a template argument of a class template, used in the argument list of a function.

```
template <template <class> auto V, template <class> concept C>
struct A {}; // A takes a variable template template argument

template <template <class T> auto V, template <class> concept C>
void foo(A<V, C>); // can accept any specialization of A; V and C are deduced

template <class T>
auto Var = 0;

template <class T>
concept Concept = true;

void test() {
    foo(A<Var, Concept>{});
}
```

[\[Run this example on Compiler Explorer\]](#)

Partial ordering of function templates involving concept template parameters

Let us introduce three concepts that refine each other A, B, and C, as well as a class template S that carries a concept X and a type T.

If we then define an overload set of two functions where one deduces the concept, we get into an interesting situation where, if concept parameters were allowed participate in partial ordering, the choice of template arguments of `S` could change the subsumption order.

```
template <template <typename T> concept X, typename T>
struct S {};
template <typename T>
concept A = true;
template <typename T>
concept B = true && A<T>;
template <typename T>
concept C = true && B<T>;

template <template <typename T> concept X, typename T>
int answer(S<X, T>) requires B<T> { return 42; }
template <template <typename T> concept X, typename T>
int answer(S<X, T>) requires X<T> { return 43; }

answer(S<A, int>{});
answer(S<C, int>{});
answer(S<B, int>{});
```

In a previous version of this proposal, we proposed that the concept template argument (`A`, `B`, `C` respectively) would be substituted in each viable `answer` overload before determining partial ordering.

However, historically, it was always possible to determine the partial ordering of two function templates before substitution, and independently of any template argument. This has notably allowed compilers to cache partial orderings of function templates, and even though the compiler isn't confused, one might legitimately be concerned that the users might be. On the face of it, it seems valuable for a C++ programmer to be able to partially order function templates in their head, and this feature seems to allow a corner-case where that is impossible before substitution.

It was always the position of the authors that use cases where concepts are deduced from functions arguments were contrived but we did not want to outright limit the set of places where a concept template parameter could be used, and it took us a while to find a reasonable way to resolve these opposite design goals.

Ultimately we found a solution that preserves all the uses cases this feature was designed for, while not making partial ordering dependent on arguments.

The rule we are proposing is:

Given a template declaration `D` with a concept parameter `C`, if `C` appears in the associated constraints of `D`, then `D` is never at least as constrained as another constrained declaration. In the example above, the 3 calls to `answer` are, with is rule, ambiguous.

This rule makes any overload that references a concept template parameter in its `requires` clause unorderable solely based on subsumption.

We think this has nice properties:

- It's fairly straightforward to teach
- It's easy to produce a good diagnosis for.
- It leaves the design space open.

Consider this slightly different example:

```
template <template <typename> concept C>
concept A = C<int>;
template <template <typename> concept C>
concept B = true && A<C>;

template <template <typename T> concept X>
void f() {}; // #1
template <template <typename T> concept X>
void f() requires A<X> {}; // #2
template <template <typename T> concept X>
void f() requires B<X> {}; // #3

template <typename T>
concept Foo = true;

f<Foo>(); // #4 (ambiguous between 2 and 3)
```

Here, #2 and #3 are more specialized than #1 (because they are constrained and #1 is not).

With the rule proposed above, neither #2 or #3 are as least as constrained as each other (as they refer to a concept template parameter x). As such #2 is not more specialized than #3 and #3 is not more specialized than #2, and the call #4 is ambiguous.

We could conceive an alternative design instead, such that we would **consider dependent concept-id (ie dependent on a concept template parameter of the function template) to be atomic constraints** (option 2).

With that alternative design, for the example above the associated constraints of #2 would be, after normalization $C<int>$ (where $C<int>$ is an atomic constraint and C refers to some invented template argument), and the associated constraints of #3 would be, after normalization $true \ \&\& \ C<int>$ (where $C<int>$ is the same expression as #2's).

In that model, #3 subsumes #2 and the call is not ambiguous. The key observation is that, the nature of C does not affect subsumption whether it would be substituted or not.

Not looking at template arguments (and considering dependent concept-id) can lead to situations where overloads are ambiguous, when they would not be if the concept argument was written verbatim and not passed via a parameter.

```
template <typename T>
concept Foo = true;

template <template <typename> concept C>
concept B = true && C<int>;
```

```
template <template <typename T> concept X>
void f() requires Foo<int>{}; // #1
```

```
template <template <typename T> concept X>
void f() requires B<X> {} // #2
```

```
f<Foo>(); // ambiguous between #1 and #2
```

The opposite is not possible.

There is a compromise between these two options. We could consider that a concept that appears (either as concept-id, or as concept template argument of another concept-id) in a subexpression of && or || makes that subexpression atomic (and that subexpression only).

```
template <template <typename> concept X>
concept AlwaysTrue = true; // X is not used
template <typename T>
concept A = true;
template <typename T, template <typename> concept C>
void f(T) requires
A<T>
|| C<int> // atomic (depends on C)
|| AlwaysTrue<T, C> {} // atomic (depends on C, even if C is never used by AlwaysTrue)
```

This would be less restrictive than Option 1, and less precise than Option 2, but easier to implement. Lets refer to this option as 1B.

In no case do we expand concept template arguments when considering subsumption; the question is merely about how much subsumption depth we want to preserve, that is, how much rope for resolving ambiguity do we want to give users.

Ultimately, while we have a slight preference for option 1.

In the previous revision of this paper, EWG was asked to choose between these options:

- Option 1: Don't try to determine a more constrained overload at all in the presence of a referenced concept template parameter.
- Option 1B: Before normalization (ie at the top level), if a concept template parameter is referenced in the subexpression of of a logical && or ||, consider that subexpression atomic
- Option 2: After normalization of non-dependent concept-id, consider concept-id referring to a concept template parameter to be atomic constraints. (Option 1 and 1B can be evolved into option 2 later, the opposite would be a breaking change.)

EWG chose the first option.

After CWG review in St Louis and [concertation on the EWG reflector](#), the same logic is applied to concept template parameters of enclosing templates, with the same motivation.

```
template <typename T>
concept A = true;
```

```

template <typename T>
concept B = A<T> && true;

template <template <typename> concept Class1,
         template <typename> concept Class2>
struct S {
    static constexpr int g(Class1 auto) {return 1;} // # 5
    static constexpr int g(Class2 auto) {return 2;} // # 6
};
static_assert(S<A, B>::g(1) == 2); // ambiguous?

```

Deduction of template parameters from the argument list of a variable template argument

This is not proposed.

Consider:

```

template<template <typename...> auto, auto>
inline constexpr bool is_specialization_of_v = false;

template<
template <typename...> auto v,
typename... Args
>
inline constexpr bool is_specialization_of_v<v, v<Args...>> = true; // #2

template <typename T>
constexpr int i = 42;

static_assert(is_specialization_of_v<i, i<int>>>); // #3

```

[\[Compiler Explorer\]](#)

Should we be able to deduce Args from int? Some existing implementations will eagerly substitute i<int> by its value (here, 42), such that there is subsequently nothing left to deduce Args against.

While it would be possible to make that work, the implementation effort is non-negligible and the benefits limited, as we could only deduce the arguments of entities that are valid template arguments - which sounds obvious but that means that the above example can only work on a subset of variables (constexpr variables template specialization of structural types).

We would also need to decide whether is_specialization_of_v<i, i<int>>> behaves differently from is_specialization_of_v<i, (i<int>>>) and how that generalizes to arbitrary subexpressions involving variable template specializations.

So, for now, arguments of variable template template parameters are not deduced. instead, we should make #2 ill-formed, so that we have the opportunity to extend that at a later time if we find sufficient motivation for it.

There are existing cases where we make non-deductible partial specializations ill-formed (see [\[temp.spec.partial.match\]](#)), however in the general case we don't seem to ([for example here is an example with a non-deducible pack](#))

Equivalence of atomic constraints

One interesting concept to consider is `tuple_of`, which would e.g., allow constraining a function on a *tuple-like* of integrals, a frequent use case in scientific computation.

In the absence of member and alias packs, a `tuple_like` concept could look like

```
template <typename T, int N>
constexpr bool __tuple_check_elements = [] {
    if constexpr (N == 0)
        return true;
    else if constexpr(requires (T t) {
        typename std::tuple_element_t<N-1, T>;
        { std::get<N-1>(t) };
    })
        return __tuple_check_elements<T, N-1>;
    return false;
}();

template <typename T>
concept tuple_like = requires {
    typename std::tuple_size<T>::type;
} && __tuple_check_elements<T, std::tuple_size_v<T>>;
```

Here, we use a `constexpr` variable template to check the constraint on individual elements. We can trivially adapt this code to take a concept argument:

```
template <typename T, template <typename> concept C>
concept decays_to = C<std::decay_t<T>>;

template <typename T, int N, template <typename> concept C>
constexpr bool __tuple_check_elements = [] {
    if constexpr (N == 0)
        return true;
    else if constexpr(requires (T t) {
        typename std::tuple_element_t<N-1, T>;
        { std::get<N-1>(t) } -> decays_to<C>;
    })
        return __tuple_check_elements<T, N-1, C>;
    return false;
}();

template <typename T, template <typename> concept C>
concept tuple_of = requires {
    typename std::tuple_size<T>::type;
} && __tuple_check_elements<T, std::tuple_size_v<T>, C>;
```

And this works fine, but `__tuple_check_elements` is an atomic constraint, so we cannot establish a subsumption relationship for this concept.

With a sufficient number of pack features, we could probably write a concept that checks all elements with a single constraint, i.e.,

```
template <typename T, typename E, int N, template <typename> concept C>
concept __tuple_of_element = requires (T t) {
    typename std::tuple_element_t<N, T>;
    { std::get<N>(t) } -> decays_to<C>;
} && C<std::tuple_element_t<0, T>>;

template <typename T, template <typename> concept C>
concept tuple_of = requires {
    typename std::tuple_size<T>::type;
} && (__tuple_of_element<T, T::[], current_expansion_index_magic(), C> && ...);
```

But in addition to relying on imaginary features, this is pretty inefficient since ordering complexity would be proportional to the square of the number of tuple elements.

Fortunately, while checking satisfaction does require looking at every element, we can look at just one element to establish subsumption in this particular case.

We can rewrite our concept as

```
template <typename T, int N, template <typename> concept C>
concept __tuple_of_element = requires (T t) {
    typename std::tuple_element_t<N, T>;
    { std::get<N>(t) } -> decays_to<C>;
} && C<std::tuple_element_t<0, T>>;

template <typename T, int N, template <typename> concept C>
constexpr bool __check_tuple_elements = [] {
    if constexpr (N == 1)
        return true;
    else if constexpr (__tuple_of_element<T, N-1, C>)
        return __check_tuple_elements<T, N-1, C>;
    return false;
}();

template <typename T, template <typename> concept C>
concept tuple_of = requires {
    typename std::tuple_size<T>::type;
} && (std::tuple_size_v<T> == 0 || (
    // Check the first element with a concept to establish subsumption
    __tuple_of_element<T, 0, C> &&
    // Check constraint satisfaction for subsequent elements
    __check_tuple_elements<T, std::tuple_size_v<T>, C>
));
```

[\[Run this example on Compiler Explorer\]](#)

For this to work, the concept template-parameter `C` needs to be substituted in the concept

`__tuple_of_element` but not in the atomic constraint `__check_tuple_elements<T, std::tuple_size_v<T>, C>`.

Atomic constraints also need to ignore concept template-parameters for the purpose of comparing their template arguments when establishing atomic constraint equivalence during subsumption.

Status of this proposal and further work

Our main priority should be to make progress on some form of universal template parameters.

This paper has been implemented in an experimental version of clang, available on godbolt.

Before that, we need to ensure concepts and variable-template template-parameters are supported features so that universal template-parameters support the gamut of entities that could reasonably be used as template-parameters.

As part of that, subsumption for concept template-parameters, as proposed in this paper, as well as subsumption of fold expressions should be considered an integral part of the design since adding them later might be somewhat challenging, although it should not affect existing valid code.

Implementation

The paper as proposed has been implemented in a fork of Clang and is available on compiler-explorer. The implementation revealed no particular challenge. In particular, we confirmed that the proposed changes do not prevent memoization for subsumption and satisfiability, i.e., a concept and the set of its concept parameters are what needs to be cached.

Wording



Preamble

[basic.pre]

Every name is introduced by a *declaration*, which is a

- *name-declaration*, *block-declaration*, or *member-declaration* [dcl.pre,class.mem],
- *init-declarator* [dcl.decl],
- *identifier* in a structured binding declaration [dcl.struct.bind],
- *init-capture* [expr.prim.lambda.capture],
- *condition* with a *declarator* [stmt.pre],
- *member-declarator* [class.mem],
- *using-declarator* [namespace.udecl],

- *parameter-declaration* [dcl.fct], [\[temp.param\]](#),
- *type-parameter* [temp.param],
- *type-tt-parameter* [temp.param],
- *variable-tt-parameter* [temp.param],
- *concept-tt-parameter* [temp.param],
- *elaborated-type-specifier* that introduces a name [dcl.type.elab],
- *class-specifier* [class.pre],
- *enum-specifier* or *enumerator-definition* [dcl.enum],
- *exception-declaration* [except.pre], or
- implicit declaration of an injected-class-name [class.pre].

◆ **Template parameter scope** **[basic.scope.temp]**

Each ~~template~~ *template-parameter* [type-tt-parameter](#), [variable-tt-parameter](#), and [concept-tt-parameter](#) introduces a *template parameter scope* that includes the *template-head* of the *template-parameter*.

◆ **Argument-dependent name lookup** **[basic.lookup.argdep]**

When the *postfix-expression* in a function call[expr.call] is an *unqualified-id*, and unqualified lookup[basic.lookup.unqual] for the name in the *unqualified-id* does not find any

- declaration of a class member, or
- function declaration inhabiting a block scope, or
- declaration not of a function or function template

then lookup for the name also includes the result of *argument-dependent lookup* in a set of associated namespaces that depends on the types of the arguments (and for [type](#) template arguments, the namespace of the template argument), as specified below.

[...]

For each argument type T in the function call, there is a set of zero or more *associated entities* to be considered. The set of entities is determined entirely by the types of the function arguments (and any ~~template~~ [type](#) [template](#) template arguments). Any *typedef-name*s and *using-declarations* used to specify the types do not contribute to this set. The set of entities is determined in the following way:

- If T is a fundamental type, its associated set of entities is empty.
- If T is a class type (including unions), its associated entities are: the class itself; the class of which it is a member, if any; and its direct and indirect base classes. Furthermore, if T is

a class template specialization, its associated entities also include: the entities associated with the types of the template arguments provided for template type parameters; the templates used as [type](#) template template arguments; and the classes of which any member templates used as [type](#) template template arguments are members. [*Note*: Constant template arguments, [variable template template arguments](#), and [concept template arguments](#) do not contribute to the set of associated entities. — *end note*]

- If *T* is an enumeration type, its associated entities are *T* and, if it is a class member, the member's class.
- If *T* is a pointer to *U* or an array of *U*, its associated entities are those associated with *U*.
- If *T* is a function type, its associated entities are those associated with the function parameter types and those associated with the return type.
- If *T* is a pointer to a member function of a class *X*, its associated entities are those associated with the function parameter types and return type, together with those associated with *X*.
- If *T* is a pointer to a data member of class *X*, its associated entities are those associated with the member type together with those associated with *X*.

In addition, if the argument is an overload set or the address of such a set, its associated entities are the union of those associated with each of the members of the set, i.e., the entities associated with its parameter types and return type. Additionally, if the aforementioned overload set is named with a *template-id*, its associated entities also include its [type](#) template arguments and those associated with its type template arguments.



The typedef specifier

[dcl.typedef]

A *simple-template-id* is only a *typedef-name* if its *template-name* names an alias template or a [type](#) template template parameter. [*Note*: [...] — *end note*]



Functions

[dcl.fct]

An *abbreviated function template* is a function declaration that has one or more generic parameter type placeholders [dcl.spec.auto]. An abbreviated function template is equivalent to a function template [temp.fct] whose *template-parameter-list* includes one invented ~~type template-parameter~~ [type-parameter](#) for each generic parameter type placeholder of the function declaration, in order of appearance. For a *placeholder-type-specifier* of the form *auto*, the invented parameter is an unconstrained *type-parameter*. For a *placeholder-type-specifier* of the form *type-constraint auto*, the invented parameter is a *type-parameter* with that *type-constraint*. The invented ~~type template-parameter~~ [type-parameter declares](#) a template parameter pack if the corresponding *parameter-declaration* declares a function parameter pack. If the placeholder contains `decltype(auto)`, the program is ill-formed. The adjusted function parameters of an abbreviated function template are derived from the *parameter-declaration-clause* by replacing each occurrence of a placeholder with the name of the corresponding invented ~~template-parameter~~ [type-parameter](#). [*Example*: [...] — *end example*]

An abbreviated function template can have a *template-head*. The invented *template-parameters* *type-parameters* are appended to the *template-parameter-list* after the explicitly declared *template-parameters*. [Example: [...] — end example]



Template parameters

[temp.param]

The syntax for *template-parameters* is:

```

template-parameter:
    type-parameter
    parameter-declaration
    type-tt-parameter
    variable-tt-parameter
    concept-tt-parameter

type-parameter:
    type-parameter-key ...opt identifieropt
    type-parameter-key identifieropt = type-id
    type-constraint ...opt identifieropt
    type-constraint identifieropt = type-id
    template-head type-parameter-key ...opt identifieropt
    template-head type-parameter-key identifieropt = id-expression

type-parameter-key:
    class
    typename

type-constraint:
    nested-name-specifieropt concept-name
    nested-name-specifieropt concept-name < template-argument-listopt >
    nested-name-specifieropt concept-name
    nested-name-specifieropt concept-name < template-argument-listopt >

type-tt-parameter:
    template-head type-parameter-key ...opt identifieropt
    template-head type-parameter-key identifieropt type-tt-parameter-defaultopt

type-tt-parameter-default:
    = nested-name-specifieropt template-name
    = nested-name-specifier template template-name

variable-tt-parameter:
    template-head auto ...opt identifieropt
    template-head auto identifieropt = nested-name-specifieropt template-name

concept-tt-parameter:
    template < template-parameter-list > concept ...opt identifieropt
    template < template-parameter-list > concept identifieropt = nested-name-specifieropt
    template-name
  
```

The component names of a *type-constraint* are its *concept-name* and those of its *nested-name-specifier* (if any).

[Note: The > token following the *template-parameter-list* of a *type-parameter* *type-tt-parameter*,

variable-tt-parameter, or concept-tt-parameter can be the product of replacing a >> token by two consecutive > tokens [temp.names]. — *end note*]

A template parameter is of one of the following kinds:

- A *type template parameter* is a template parameter introduced by a *type-parameter*.
- A *constant template parameter* is a template parameter introduced by a *parameter-declaration*.
- A *type template template parameter* is a template parameter introduced by a *type-tt-parameter*.
- A *variable template template parameter* is a template parameter introduced by a *variable-tt-parameter*.
- A *concept template parameter* is a template parameter introduced by a *concept-tt-parameter*.

Type template template parameters, variable template template parameters, and concept template parameters are collectively referred to as *template template parameters*.

A concept template parameter shall not have associated constraints [temp.constr.decl].

[*Editor's note*: Paragraph 17 is modified and transplanted here to appear immediately after the above paragraph.]

If a *template-parameter* is a ~~*type-parameter with an ellipsis prior to its optional identifier or is a parameter-declaration*~~ that declares a pack [dcl.fct], or otherwise has an ellipsis prior to its optional identifier, then the *template-parameter* is declares a template parameter pack [temp.variadic]. A template parameter pack that is a *parameter-declaration* whose type contains one or more unexpanded packs is a pack expansion. Similarly, a template parameter pack that is a template template parameter with a *template-parameter-list* containing one or more unexpanded packs is a pack expansion. A type parameter pack with a *type-constraint* that contains an unexpanded parameter pack is a pack expansion. A template parameter pack that is a pack expansion shall not expand a template parameter pack declared in the same *template-parameter-list*.

[*Editor's note*: Remove the footnote]

[*Footnote*: ~~Since *template template parameters* and *template template arguments* are treated as types for descriptive purposes, the terms *constant parameter* and *constant argument* are used to refer to non-type, non-template parameters and arguments.~~ — *end note*]

A *template-parameter* of the form `class identifier` is a *type-parameter*. [Example:

```
class T { /*...*/ };
int i;

template<class T, T i> void f(T t) {
    T t1 = i;           // template parameters T and i
    ::T t2 = ::i;       // global namespace members T and i
}
```

Here, the template `f` has a type template parameter called `T`, rather than an unnamed constant template parameter of class `T`. — *end example*] The *parameter-declaration* of a *template-parameter* shall not have a *storage-class-specifier*. Types shall not be defined in a template parameter declaration.

The *identifier* in a ~~type-parameter~~ *template-parameter* denoting a type or template is not looked up. ~~A type-parameter whose~~ An identifier that does not follow an ellipsis ~~defines its identifier~~ is defined to be a *typedef-name* (if declared without `template`) or *template-name* (if declared with `template`).

- a *typedef-name* for a *type-parameter*,
- a *template-name* for a *variable-tt-parameter*,
- a *template-name* for a *type-tt-parameter*, or
- a *concept-name* for a *concept-tt-parameter*,

in the scope of the template declaration.

[*Note*: A template argument can be a class template or alias template. For example,

```
template<class T> class myarray { /*...*/ };

template<class K, class V, template<class T> class C = myarray>
class Map {
    C<K> key;
    C<V> value;
};
```

— *end note*]

[*Editor's note*: Modify [temp.param]/p16 As follow]

A *template-parameter* of a template *template-parameter* is permitted to have a default template argument. When such default arguments are specified, they apply to the template *template-parameter* in the scope of the template *template-parameter*. [*Example*:

```
template <template <class TT = float> class T> struct A {
    inline void f();
    inline void g();
};
template <template <class TT> class T> void A<T>::f() {
    T<> t;           // error: TT has no default template argument
}
template <template <class TT = char> class T> void A<T>::g() {
    T<> t;           // OK, T<char>
}
```

— *end example*]

The associated constraints of a template template parameter shall not contain a concept-dependent constraint. [*Example*:

```
template <
    template <typename> concept C,
    template <C> class TT // error: C forms a concept-dependent constraint
>
struct A {};
```

— *end example*]

[...]

◆ Names of template specializations [temp.names]

A template specialization [temp.spec] can be referred to by a *template-id*:

[...]

```
template-argument:
    constant-expression
    type-id
    id-expression
    nested-name-specifieropt template-name
    nested-name-specifier template template-name
```

[...]

◆ Template arguments [temp.arg]

◆ General [temp.arg.general]

~~There are three forms of *template-argument*, corresponding to the three forms of *template-parameter*: type, constant and template.~~ The type and form of each *template-argument* specified in a *template-id* shall match the type and form specified for the corresponding parameter declared by the template in its *template-parameter-list*. [...]

[...]

◆ Type template arguments [temp.arg.type]

A *template-argument* for a ~~*template-parameter which is a type*~~ type template parameter shall be a *type-id*.

[...]

◆ Constant template arguments [temp.arg.nontype]

A template argument *E* for a constant template parameter with declared type *T* shall be such that the invented declaration

$T \ x = E ;$

satisfies the semantic constraints for the definition of a constexpr variable with static storage duration[dcl.constexpr]. If T contains a placeholder type[dcl.spec.auto] or a placeholder for a deduced class type[dcl.type.class.deduct], the type of the parameter is deduced from the above declaration. [*Note: E is a *template-argument* or (for a default template argument) an *initializer-clause*. — end note*] If the parameter type thus deduced is not permitted for a **template-parameter-declaration** constant template parameter[temp.param], the program is ill-formed.

◆ **Template template arguments** **[temp.arg.template]**

A *template-argument* for a template template parameter shall be the name of a **class** template **or an alias template, expressed as id-expression**. For a type-tt-parameter, the name shall denote a class template or alias template. For a variable-tt-parameter, the name shall denote a variable template. For a concept-tt-parameter, the name shall denote a concept. Only primary templates are considered when matching the template template argument with the corresponding parameter; partial specializations are not considered even if their parameter lists match that of the template template parameter.

Any partial specializations[temp.spec.partial] associated with the primary template are considered when a specialization based on the template template parameter is instantiated. [...]

A template template parameter P and a *template-argument* A are *compatible* if

- A denotes a class template or an alias template and P is a type template parameter,
- A denotes a variable template and P is a variable template parameter, or
- A denotes a concept and P is a concept template parameter.

[*Editor's note: See CWG2398*]

A template *template-argument* A matches a template template parameter P when A and P are compatible and P is at least as specialized as A , ignoring constraints on A if P is unconstrained. If P contains a template parameter pack, then A also matches P if each of A 's template parameters matches the corresponding template parameter declared in the *template-head* of P .

[...]

A template template parameter P is at least as specialized as a template *template-argument* A if, given the following rewrite to two function templates, the function template corresponding to P is at least as specialized as the function template corresponding to A according to the partial ordering rules for function templates[temp.func.order]. Given an invented class template X with the *template-head* of A (including default arguments and *requires-clause*, if any):

- Each of the two function templates has the same template parameters and *requires-clause* (if any), respectively, as P or A .

- Each function template has a single function parameter whose type is a specialization of X with template arguments corresponding to the template parameters from the respective function template where, for each *template-parameter* PP in the *template-head* of the function template, a corresponding *template-argument* AA is formed. If PP declares a template parameter pack, then AA is the pack expansion $PP \dots$ [temp.variadic]; otherwise, AA is an *id-expression* denoting PP .

If the rewrite produces an invalid type, then P is not at least as specialized as A .

[...]

❖ Constraints [temp.constr.constr]

A *constraint* is a sequence of logical operations and operands that specifies requirements on template arguments. The operands of a logical operation are constraints. There are **four** five different kinds of constraints:

- conjunctions[temp.constr.op],
- disjunctions[temp.constr.op],
- atomic constraints[temp.constr.atomic], **and**
- fold expanded constraints[temp.constr.fold]: , and
- concept-dependent constraints[temp.constr.concept].

[...]

[Editor's note: Add a new section after [temp.constr.atomic]]

❖ Concept-dependent constraints [temp.constr.concept]

A *concept-dependent constraint* CD is an atomic constraint whose expression is a concept-id CI whose *concept-name* names a dependent concept named C .

To determine if CD is satisfied, the parameter mapping and template arguments are first substituted into C . If substitution results in an invalid concept-id in the immediate context of the constraint ([temp.deduct.general]), the constraint is not satisfied. Otherwise, let CI' be the normal form of the concept-id after substitution of C .

[Note: Normalization of CI may be ill-formed; no diagnostics is required. — end note]

To form CI'' , each appearance of C 's template parameters in the parameter mappings of the atomic constraints (including concept-dependent constraints) in CI' is substituted with their respective arguments from the parameter mapping of CD and the arguments of CI .

The constraint is satisfied if CI'' is satisfied.

[Note: Checking whether CI'' is satisfied can lead to further normalization of concept-dependent constraints. — end note]

[Example:

```

template <typename>
concept C = true;

template <typename T, template <typename> concept CC>
concept D = CC<T>;

template <typename T,
        template <typename> concept CT,
        template <typename, template <typename> concept> concept CU
>
int f() requires CU<T, CT>;

int _ = f<int, C, D>(); // #1

```

In this example, the associated constraint of `f` is a concept-dependent constraint *CI* whose expression is the concept-id `CU<T, CT>` with the mapping $T \mapsto T, CT \mapsto CT, CU \mapsto CU$.

CI' is the result of substituting `D` into *CI*.

We consider the normal form *CI''* of `D<T, CT>`, which is `CC<T>` with the mapping $T \mapsto T, CC \mapsto CC$.

By recursion, `C` is substituted in `CC<T>` and then normalized to the atomic constraint `true`, which is satisfied.

— end example]

Constraint normalization [temp.constr.normal]

The *normal form* of an *expression* `E` is a constraint [temp.constr.constr] that is defined as follows:

- The normal form of an expression `( E )` is the normal form of `E`.
- The normal form of an expression `E1 || E2` is the disjunction [temp.constr.op] of the normal forms of `E1` and `E2`.
- The normal form of an expression `E1 && E2` is the conjunction of the normal forms of `E1` and `E2`.
- ~~The normal form of~~ For a concept-id `C<A1, A2, ..., An>` termed CI: is the normal form of the *constraint-expression* of `C`, after substituting `A1, A2, ..., An` for `C`'s respective template parameters in the parameter mappings in each atomic constraint. If any such substitution results in an invalid type or expression, the program is ill-formed; no diagnostic is required.
 - If `C` names a dependent concept, the normal form of *CI* is a concept-dependent constraint whose concept-id is *CI* and whose parameter mapping is the identity mapping.
 - Otherwise, to form *CE*, any non-dependent concept template argument `Ai` is substituted into the *constraint-expression* of `C`. If any such substitution results in an invalid concept-id, the program is ill-formed; no diagnostic is required.

The normal form of CI is the result of substituting, in the normal form N of CE, appearances of C's template parameters in the parameter mappings of the atomic constraints in N with their respective arguments from C. If any such substitution results in an invalid type or expression, the program is ill-formed; no diagnostic is required.

[Example:

```
template<typename T> concept A = T::value || true;
template<typename U> concept B = A<U*>;
template<typename V> concept C = B<V&*>;
```

Normalization of B's *constraint-expression* is valid and results in $T::value$ (with the mapping $T \mapsto U*$) $\vee$ $true$ (with an empty mapping), despite the expression $T::value$ being ill-formed for a pointer type T . Normalization of C's *constraint-expression* results in the program being ill-formed, because it would form the invalid type $V&*$ in the parameter mapping. — end example]

- For a *fold-operator* Op [expr.prim.fold] that is either $\&\&$ or $\|\bar{r}:$
 - The normal form of an expression $(\dots \text{fold-operator } \text{Op } E)$ is the normal form of $(E \text{fold-operator } \text{Op } \dots)$.
 - For a *fold-operator that is either $\&\&$ or $\|\bar{r}$* , the normal form of an expression $(E_1 \text{fold-operator } \text{Op } \dots \text{fold-operator } \text{Op } E_2)$ is the normal form of
 - ✱ $(E_1 \text{fold-operator } \text{Op } \dots) \text{fold-operator } \text{Op } E_2$ if E_1 contains an unexpanded pack, or
 - ✱ $E_1 \text{fold-operator } \text{Op } (E_2 \text{fold-operator } \text{Op } \dots)$ otherwise.
 - The normal form of an expression F of the form $(E \text{Op } \dots)$ is as follows:
If E names any unexpanded concept template parameter pack, it shall not name any unexpanded template parameter pack of another kind.
Let E' be the normal form of E .

- ✱ If E names any unexpanded concept template parameter pack P_k that has corresponding template arguments in the parameter mapping of any atomic constraint (including concept-dependent constraints) of E' , the number of arguments specified for all such P_k shall be the same number N .

The normal form of F is the normal form of $E_0 \text{Op } E_1 \text{Op } \dots \text{Op } E_{N-1}$ after substituting in E_i the respective i^{th} concept argument of each P_k . If any such substitution results in an invalid type or expression, the program is ill-formed; no diagnostic is required.

- ✱ Otherwise, the normal form of F is a fold expanded constraint [temp.constr.fold] whose constraint is E' and whose *fold-operator* is Op .

- The normal form of $(E \ \&\& \ \dots)$ is a fold expanded constraint [temp.constr.fold] whose constraint is the normal form of E and whose *fold-operator* is $\&\&$.
- The normal form of $(E \ || \ \dots)$ is a fold expanded constraint whose constraint is the normal form of E and whose *fold-operator* is $||$.
- The normal form of any other expression E is the atomic constraint whose expression is E and whose parameter mapping is the identity mapping.

The process of obtaining the normal form of a *constraint-expression* is called *normalization*. [Note: Normalization of *constraint-expressions* is performed when determining the associated constraints[temp.constr.constr] of a declaration and when evaluating the value of an *id-expression* that names a concept specialization[expr.prim.id]. — end note]

[Example:

```
template<typename T> concept C1 = sizeof(T) == 1;
template<typename T> concept C2 = C1<T> && 1 == 2;
template<typename T> concept C3 = requires { typename T::type; };
template<typename T> concept C4 = requires (T x) { ++x; };

template<C2 U> void f1(U);      // #1
template<C3 U> void f2(U);      // #2
template<C4 U> void f3(U);      // #3
```

The associated constraints of #1 are $\text{sizeof}(T) == 1$ (with mapping $T \mapsto U$) $\wedge 1 == 2$.

The associated constraints of #2 are $\text{requires } \{ \text{typename } T::\text{type}; \}$ (with mapping $T \mapsto U$).

The associated constraints of #3 are $\text{requires } (T \ x) \{ ++x; \}$ (with mapping $T \mapsto U$). — end example]

[Example:

```
template <typename T>
concept C = true;

template <typename T, template<typename> concept CT>
concept CC = CT<T>;

template <typename U,
        template<typename, template<typename> concept> concept CT>
void f() requires CT<U*, C>;

template <typename U>
void g() requires CC<U*, C>;
```

The normal form of the associated constraints of f is the concept-dependent constraint $CT<T, C>$.

The normal form of the associated constraints of g is the atomic constraint true .

— end example] [Example:

```

template <typename T>
concept A = true;
template <typename T>
concept B = A<T> && true; // B subsumes A
template <typename T>
concept C = true;
template <typename T>
concept D = C<T> && true; // D subsumes C

template <typename T, template <typename> concept... CTs>
concept all_of = (CTs<T> && ...);

template <typename T> requires all_of<T, A, C>
constexpr int f(T) { return 1; } // #1

template <typename T> requires all_of<T, B, D>
constexpr int f(T) { return 2; } // #2

static_assert(f(1) == 2); // ok

```

The normal form of `all_of<T, A, C>` is the conjunction of the normal forms of `A<T>` and `C<T>`. Similarly, the normal form of `all_of<T, B, D>` is the conjunction of the normal forms of `B<T>` and `D<T>`. #2 therefore is more constrained than #1.

— *end example*] [*Example:*

```

template <typename T, template <typename> concept>
struct wrapper{};

template <typename... T, template <typename> concept... CTs>
int f(wrapper<T, CTs>...) requires (CTs<T> && ...); // error: the fold expression expands
    mixed kind template parameters.

```

— *end example*]

Partial ordering by constraints **[temp.constr.order]**

A constraint P *subsumes* a constraint Q if and only if, for every disjunctive clause P_i in the disjunctive normal form of P , P_i subsumes every conjunctive clause Q_j in the conjunctive normal form of Q , where

- a disjunctive clause P_i subsumes a conjunctive clause Q_j if and only if there exists an atomic constraint P_{ia} in P_i for which there exists an atomic constraint Q_{jb} in Q_j such that P_{ia} subsumes Q_{jb} ,
- an atomic constraint A subsumes another atomic constraint B if and only if A and B are identical using the rules described in ??, and
- a fold expanded constraint A subsumes another fold expanded constraint B if they are compatible for subsumption, have the same *fold-operator*, and the constraint of A subsumes that of B .

[*Example:* Let A and B be atomic constraints[temp.constr.atomic]. The constraint $A \wedge B$ subsumes A , but A does not subsume $A \wedge B$. The constraint A subsumes $A \vee B$, but $A \vee B$ does not subsume A . Also note that every constraint subsumes itself. — *end example*]

[*Note:* The subsumption relation defines a partial ordering on constraints. This partial ordering is used to determine

- the best viable candidate of non-template functions[over.match.best],
- the address of a non-template function[over.over],
- the matching of template template arguments[temp.arg.template],
- the partial ordering of class template specializations[temp.spec.partial.order], and
- the partial ordering of function templates[temp.func.order].

— *end note*]

The associated constraints C of a declaration D are *eligible for subsumption* unless C contains a concept-dependent constraint.

A declaration $D1$ is *at least as constrained* as a declaration $D2$ if

- $D1$ and $D2$ are both constrained declarations and $D1$'s associated constraints [are eligible for subsumption and](#) subsume those of $D2$; or
- $D2$ has no associated constraints.

A declaration $D1$ is *more constrained* than another declaration $D2$ when $D1$ is at least as constrained as $D2$, and $D2$ is not at least as constrained as $D1$. [*Example:*

```
template<typename T> concept C1 = requires(T t) { --t; };
template<typename T> concept C2 = C1<T> && requires(T t) { *t; };

template<C1 T> void f(T);           // #1
template<C2 T> void f(T);           // #2
template<typename T> void g(T);    // #3
template<C1 T> void g(T);           // #4

f(0);                             // selects #1
f((int*)0);                        // selects #2
g(true);                          // selects #3 because C1<bool> is not satisfied
g(0);                             // selects #4
```

— *end example*]

[*Example:*

```
template <template <typename T> concept CT, typename T>
struct S {};
template <typename T>
concept A = true;

template <template <typename T> concept X, typename T>
```

```
int f(S<X, T>) requires A<T> { return 42; } // #1
template <template <typename T> concept X, typename T>
int f(S<X, T>) requires X<T> { return 43; } // #2

f(S<A, int>{}); // ok, select #1 because #2 is not eligible for subsumption
```

— *end example*]



Variadic templates

[temp.variadic]

A *pack expansion* consists of a *pattern* and an ellipsis, the instantiation of which produces zero or more instantiations of the pattern in a list (described below). The form of the pattern depends on the context in which the expansion occurs. Pack expansions can occur in the following contexts:

- In a function parameter pack [dcl.fct]; the pattern is the *parameter-declaration* without the ellipsis.
- In a *using-declaration* [namespace.udecl]; the pattern is a *using-declarator*.
- In a template parameter pack that is a pack expansion:
 - if the template parameter pack is a *parameter-declaration*; the pattern is the *parameter-declaration* without the ellipsis;
 - if the template parameter pack is a *type-parameter*; the pattern is the corresponding *type-parameter* without the ellipsis.
 - if the template parameter pack is a template template parameter; the pattern is the corresponding *type-tt-parameter*, *variable-tt-parameter*, or *concept-tt-parameter* without the ellipsis.
- [...]

[...]

The instantiation of a pack expansion considers items $E_1, E_2, \dots, E_N$, where N is the number of elements in the pack expansion parameters. Each E_i is generated by instantiating the pattern and replacing each pack expansion parameter with its i^{th} element. Such an element, in the context of the instantiation, is interpreted as follows:

- if the pack is a template parameter pack, the element is *an id-expression (for a constant template parameter pack), a typedef-name (for a type template parameter pack declared without template), or a template-name (for a type template parameter pack declared with template)*,
 - an *id-expression* for a constant template parameter pack,
 - a *typedef-name* for a type template parameter pack, or
 - a *template-name* for a template template parameter pack

designating the i^{th} corresponding type, template, or constant template argument;

- [...]

[...]

◆ Function template overloading

[temp.over.link]

[...]

Two *template-heads* are *equivalent* if their *template-parameter-lists* have the same length, corresponding *template-parameters* are equivalent and are both declared with *type-constraint* *s* that are equivalent if either *template-parameter* is declared with a *type-constraint*, and if either *template-head* has a *requires-clause*, they both have *requires-clauses* and the corresponding *constraint-expressions* are equivalent. Two *template-parameters* are *equivalent* under the following conditions:

- they declare template parameters of the same kind,
- if either declares a template parameter pack, they both do,
- if they declare constant template parameters, they have equivalent types ignoring the use of *type-constraints* for placeholder types, and
- if they declare template template parameters, their **template-parameters** kinds are the same and their *template-heads* are equivalent.

[...]

◆ Partial ordering of function templates

[temp.func.order]

If multiple function templates share a name, the use of that name can be ambiguous because template argument deduction [temp.deduct] may identify a specialization for more than one function template. *Partial ordering* of overloaded function template declarations is used in the following contexts to select the function template to which a function template specialization refers:

- during overload resolution for a call to a function template specialization [over.match.best];
- when the address of a function template specialization is taken;
- when a placement operator delete that is a function template specialization is selected to match a placement operator new [basic.stc.dynamic.deallocation,expr.new];
- when a friend function declaration [temp.friend], an explicit instantiation [temp.explicit] or an explicit specialization [temp.expl.spec] refers to a function template specialization.

Partial ordering selects which of two function templates is more specialized than the other by transforming each template in turn (see next paragraph) and performing template argument deduction using the function type. The deduction process determines whether one of the templates is more specialized than the other. If so, the more specialized template is the one

chosen by the partial ordering process. If both deductions succeed, the partial ordering selects the more constrained template (if one exists) as determined below.

To produce the transformed template, for each type, constant, ~~or template~~ type template, variable template, or concept template parameter (including template parameter packs [temp.variadic] thereof) synthesize a unique type, value, ~~or~~ class template, variable template, or concept, respectively, and substitute it for each occurrence of that parameter in the function type of the template.

A synthesized template has the same *template-head* as its corresponding template template parameter.

[...]

◆ Locally declared names [temp.local]

Like normal (non-template) classes, class templates have an injected-class-name[class.pre]. The injected-class-name can be used as a *template-name* or a *type-name*. When it is used with a *template-argument-list*, as a *template-argument* for a type template template parameter, or as the final identifier in the *elaborated-type-specifier* of a friend class template declaration, it is a *template-name* that refers to the class template itself. Otherwise, it is a *type-name* equivalent to the *template-name* followed by the template argument list[temp.decls.general,temp.arg.general] of the class template enclosed in <>.

[...]

◆ Dependent types [temp.dep.type]

[...]

A template argument that is equivalent to a template parameter can be used in place of that template parameter in a reference to the current instantiation. ~~For a template type-parameter, a~~ A template argument is equivalent to a type template parameter if it denotes the same type. ~~For a constant template-parameter, a~~ A template argument is equivalent to a constant template parameter if it is an *identifier* that names a variable that is equivalent to the template parameter. [...]

A type is dependent if it is

- [...]
- denoted by a *simple-template-id* in which either the template name is a template parameter or any of the template arguments ~~is a dependent type or an expression that is type-dependent or value-dependent or is a pack expansion~~ is dependent [temp.dep.temp].
[Footnote: [...] — end note]
- [...]

[...]

◆ Value-dependent expressions

[temp.dep.constexpr]

[...]

An *id-expression* is value-dependent if:

- it is a concept-id and [its *concept-name* is dependent or](#) any of its arguments are dependent [\[temp.dep.temp\]](#),
- it is type-dependent,
- it is the name of a constant template parameter,
- [...]

[...]

template-parameters in the *template-parameter-list* shall be the same as those specified in the primary template definition. [Example: [...] — end example]

[...]

◆ Dependent template arguments

[temp.dep.temp]

A type *template-argument* is dependent if the type it specifies is dependent.

A constant *template-argument* is dependent if its type is dependent or the constant expression it specifies is value-dependent.

Furthermore, a constant *template-argument* is dependent if the corresponding constant template parameter is of reference or pointer type and the *template-argument* designates or points to a member of the current instantiation or a member of a dependent type.

[A template argument is also dependent if it is a pack expansion.](#)

A template template parameter is dependent if it names a template parameter or its terminal name is dependent.

[...]

◆ Template argument deduction

[temp.deduct]

◆ Deducing template arguments from a type

[temp.deduct.type]

[Editor's note: Modify [temp.deduct.type]/p8 As follow]

A [type](#) template **type** argument T, a template template argument TT [denoting a class template or an alias template](#), [a template template argument VV denoting a variable template or a concept](#), or a constant template argument i can be deduced if P and A have one of the following forms:

$^{CV_{opt}} T$
 T^*
 $T\&$

```

T&&
Topt [iopt ]
Topt (Topt ) noexcept(iopt )
Topt Topt ::*
TTopt <T>
TTopt <i>
TTopt <TT>
TTopt <VV>
TTopt <>

```

where

- T_{opt} represents a type or parameter-type-list that either satisfies these rules recursively, is a non-deduced context in P or A, or is the same non-dependent type in P and A,
- TT_{opt} represents either a class template or a template template parameter,
- i_{opt} represents an expression that either is an i, is value-dependent in P or A, or has the same constant value in P and A, and
- noexcept(i_{opt}) represents an exception specification[except.spec] in which the (possibly-implicit, see ??) *noexcept-specifier's* operand satisfies the rules for an i_{opt} above.

[*Note*: If a type matches such a form but contains no Ts, is, or TTs, deduction is not possible. — *end note*]

Similarly, ~~<T> represents template argument lists where at least one argument contains a T, <i> represents template argument lists where at least one argument contains an i~~ <X> represents template argument lists where at least one argument contains an X, where X is one of T, i, TT, or VV; and <> represents template argument lists where no argument contains a T, ~~an i, a TT, or a VV~~ or an i.

If P has a form that contains ~~<T> or~~ <i>, <TT>, or <VV>, then each argument P_i of the respective template argument list of P is compared with the corresponding argument A_i of the corresponding template argument list of A. If the template argument list of P contains a pack expansion that is not the last template argument, the entire template argument list is a non-deduced context. If P_i is a pack expansion, then the pattern of P_i is compared with each remaining argument in the template argument list of A.

Feature test macro

[*Editor's note*: Add a new macro in [tab:cpp.predefined.ft] : __cpp_template_parameters set to the date of adoption] .

Acknowledgments

Many people contributed valuable discussions and feedbacks to this paper, notably Brian Bi, Nina Dinka Ranns, Alisdair Meredith, Joshua Berne, Pablo Halpern, Lewis Baker, Bengt Gustafsson, Hannes Hauswedell, Richard Smith and Barry Revzin.

We would like to thanks Daveed Vandevoorde for convince us to find a design that does not affect partial ordering.

We also want to thank Lori Hughes for helping editing this paper and Bloomberg for sponsoring this work.

References

- [1] Gašper Ažman, Mateusz Pusz, Colin MacLean, Bengt Gustafsonn, and Corentin Jabot. P1985R3: Universal template parameters. <https://wg21.link/p1985r3>, 9 2022.
- [2] Jorg Brown. P1715R1: Loosen restrictions on “_t” typedefs and “_v” values. <https://wg21.link/p1715r1>, 2 2023.
- [3] Peter Gottschling. N3595: Simplifying argument-dependent lookup rules. <https://wg21.link/n3595>, 3 2013.
- [4] Corentin Jabot. P2963R0: Ordering of constraints involving fold expressions. <https://wg21.link/p2963r0>, 9 2023.
- [5] Corentin Jabot, Pablo Halpern, John Lakos, Alisdair Meredith, Joshua Berne, and Gašper Ažman. P2632R0: A plan for better template meta programming facilities in c++26. <https://wg21.link/p2632r0>, 10 2022.
- [6] Mateusz Pusz. P2008R0: Enable variable template template parameters. <https://wg21.link/p2008r0>, 1 2020.
- [7] Gabriel Dos Reis. N3615: Constexpr variable templates. <https://wg21.link/n3615>, 3 2013.
- [8] Herb Sutter. P0934R0: A modest proposal: Fixing adl. <https://wg21.link/p0934r0>, 2 2018.